Détection et Génération d'Émotions par Contexte Visuel de Scène

Pipeline YOLO + Architecture Multi-Branches sur EMOTIC

---

1. Contexte

L'Affective Computing s'est longtemps appuyé sur l'analyse du visage isolé pour inférer les émotions humaines. Or, notre perception émotionnelle est profondément contextuelle : une personne assise sur un canapé dans un salon ne suscite pas la même lecture qu'une personne assise sur un trottoir de nuit. Les travaux de Kosti et al. (2017, 2019) ont mis en évidence ce verrou en montrant que le contexte visuel — objets, lieu, posture corporelle — contribue significativement à la reconnaissance émotionnelle. Nous nous inscrivons dans cette lignée en proposant une pipeline qui exploite explicitement la sémantique des objets détectés dans la scène. Les applications sont concrètes : robotique sociale (interpréter l'état affectif d'un utilisateur en situation réelle), analyse automatique de contenus médiatiques, génération d'environnements émotionnellement expressifs pour le jeu vidéo ou la réalité virtuelle, et outils d'accessibilité pour les personnes ayant des difficultés à décoder les signaux émotionnels.

---

2. Problématique

Nous cherchons à répondre à une double question : *comment prédire les émotions ressenties par une personne à partir de la scène visuelle complète qui l'entoure, et comment générer des scènes dont le registre émotionnel est contrôlé ?*

Le premier défi est un problème de classification multi-label sur 26 catégories fortement déséquilibrées, couplé à une régression continue sur les dimensions Valence-Arousal-Dominance (VAD). L'ambiguïté est intrinsèque : une même image peut légitimement être annotée « anticipation » et « bonheur », et les annotateurs humains divergent fréquemment. Le second défi — la génération conditionnelle — est encore plus ouvert : modifier l'émotion perçue d'une scène entière ne se réduit pas à une manipulation faciale ; il faut agir sur la lumière, les objets, l'atmosphère globale.

Notre valeur ajoutée repose sur l'intégration de YOLOv8 comme extracteur de sémantique contextuelle explicite. Plutôt que de laisser un backbone CNN découvrir implicitement que « couteau » et « bouquet de fleurs » ne portent pas la même charge émotionnelle, nous fournissons au modèle un vecteur structuré d'objets détectés, avec leurs scores de confiance. Ce signal contextuel, fusionné avec les features du corps et de la scène globale, doit permettre un gain de précision interprétable sur les catégories les plus ambiguës.

---

3. Description des Données

Dataset	URL	Nb images	Variables clés	Nature des variables
EMOTIC	http://sunai.uoc.edu/emotic	~23 571 images, ~34 320 annotations (plusieurs personnes/image)	26 catégories discrètes d'émotions, 3 dimensions continues VAD (Valence, Arousal, Dominance), bounding boxes des personnes, image complète de la scène	Catégories : binaire multi-label (0/1 par catégorie). VAD : continu [0–10]. Bounding boxes : coordonnées entières (pixels)
COCO 2017	https://cocodataset.org	~118k (train) + 5k (val) — *non téléchargé intégralement*	80 classes d'objets, bounding boxes, segmentation	Détection d'objets : classes catégorielles, bounding boxes continues
AffectNet (sous-ensemble)	http://mohammadmahoor.com/affectnet/	~450k annotations (nous n'en utiliserions qu'un sous-ensemble de ~30k crops de visages)	8 catégories d'expressions faciales, valence et arousal continus	Catégoriel + continu

Dataset	URL	Nb images	Variables clés	Nature des variables

Commentaires sur les données. Le split officiel d'EMOTIC (train/val/test) est respecté pour assurer la comparabilité avec la littérature. Le déséquilibre de classes est marqué : des catégories comme « Engagement » ou « Anticipation » sont largement majoritaires, tandis que « Aversion », « Suffering » ou « Sensitivity » sont sous-représentées par un facteur 10 ou plus. Nous envisageons deux stratégies complémentaires pour y répondre : un ré-échantillonnage pondéré dans le DataLoader (oversampling des classes rares) et l'utilisation d'une Focal Loss en remplacement ou en complément de la BCE, afin de pénaliser davantage les erreurs sur les classes peu fréquentes.

Il faut aussi reconnaître la subjectivité inhérente aux annotations émotionnelles : les labels EMOTIC reflètent un consensus d'annotateurs Amazon Mechanical Turk, ce qui introduit un bruit irréductible. Nous ne cherchons pas à le gommer, mais à le prendre en compte dans l'interprétation de nos métriques — un mAP de 0.30 sur EMOTIC est déjà un résultat significatif, la baseline de Kosti et al. se situant autour de 0.27.

---

4. Méthodologie Envisagée

Composant 1 — Détection d'objets contextuels (YOLOv8)

Architecture. Nous utilisons YOLOv8-medium (`yolov8m.pt`) de la librairie Ultralytics, chargé

avec ses poids pré-entraînés sur COCO (80 classes). Le choix du variant « medium » est un compromis entre capacité de détection et contrainte VRAM : le modèle tient en ~50 Mo et l'inférence sur une image 640×640 prend environ 15–25 ms sur GPU T4. Le variant « large » apporte un gain marginal en mAP COCO (+1–2 points) pour un coût mémoire nettement supérieur, ce qui n'est pas justifié ici puisque YOLO sert de feature extractor, pas de détecteur haute précision.

Extraction des features. Pour chaque image EMOTIC, nous effectuons un forward pass YOLOv8 et construisons un vecteur contextuel de dimension 80 (une entrée par classe COCO). Chaque entrée contient le score de confiance maximal parmi toutes les détections de cette classe dans l'image, ou 0 si la classe est absente. Ce vecteur est donc un « profil d'objets » de la scène : [personne: 0.94, canapé: 0.87, livre: 0.72, …, zèbre: 0.0]. Pour les classes détectées plusieurs fois (ex : plusieurs personnes), nous conservons le max des scores plutôt que la somme, afin d'éviter un biais de magnitude.

Pré-calcul. Ces vecteurs peuvent être pré-calculés offline pour l'ensemble du dataset EMOTIC (inférence ~23k images × 20 ms ≈ 8 minutes sur T4). Ils sont stockés en tant que tenseurs `.pt` et chargés pendant l'entraînement du Composant 2, ce qui dissocie complètement le coût YOLO du budget d'entraînement.

Librairies. `ultralytics` (pip install ultralytics), PyTorch pour la manipulation des tenseurs.

Métrique. Nous ne ré-entraînons pas YOLO, donc pas de métrique de détection propre. Néanmoins, nous rapportons des statistiques descriptives sur les détections : nombre moyen d'objets par image EMOTIC, distribution des classes détectées, taux d'images sans objet détecté (hors personne).

Composant 2 — Détection d'émotions (architecture multi-branches)

Architecture globale. Le modèle de détection s'articule en trois branches parallèles dont les représentations sont fusionnées avant la tête de prédiction :

Branche 1 — Corps/Personne. Le crop de la personne (issu du bounding box fourni par EMOTIC) est redimensionné à 224×224 et passé dans un backbone EfficientNet-B2 pré-entraîné sur ImageNet. Nous avons écarté ResNet-50 malgré sa popularité car EfficientNet-B2 offre des performances comparables sur ImageNet avec ~3× moins de paramètres (≈9M vs ~25M), ce qui est déterminant sur Colab T4 où la mémoire est partagée avec les deux autres branches. La sortie est le vecteur de features avant la couche de classification (1408 dimensions).

Branche 2 — Contexte global. L'image complète est redimensionnée à 224×224 et traitée par un second backbone (ResNet-18 pré-entraîné ImageNet, choisi ici pour sa légèreté : ~11M paramètres). L'objectif n'est pas d'extraire des features fines — c'est le rôle de la Branche 1 — mais de capturer la structure spatiale et l'ambiance de la scène (intérieur/extérieur, luminosité, encombrement). Sortie : 512 dimensions.

Branche 3 — Features YOLO. Le vecteur de 80 dimensions produit par le Composant 1 est passé dans un petit MLP (80 → 128 → 64, avec BatchNorm et ReLU) pour projeter les features

catégorielles des objets dans un espace de dimension compatible avec les branches visuelles. Ce MLP est entraîné conjointement avec le reste du modèle.

Fusion. Les trois vecteurs (1408 + 512 + 64 = 1984 dimensions) sont concaténés et alimentent une tête de prédiction composée de deux couches FC avec Dropout (p=0.3) :

- Sortie 1 : 26 logits pour la classification multi-label (BCE Loss, ou Focal Loss si le déséquilibre l'exige)
- Sortie 2 : 3 neurones pour la régression VAD (MSE Loss)
- Loss totale : `L = L_BCE + λ · L_MSE` avec λ ≈ 0.5 (à calibrer par validation)

Entraînement. Optimiseur : AdamW, lr = 1e-4 avec cosine annealing sur 25 époques. Les backbones sont d'abord gelés pendant 5 époques (warm-up de la tête de fusion), puis dégélés avec un lr réduit (1e-5) pour fine-tuning. Batch size estimé : 32 (à vérifier empiriquement sur T4 — le modèle complet pèse ~20M de paramètres, les crops + images complètes doublent l'empreinte mémoire). Data augmentation via Albumentations : horizontal flip, rotation ±15°, color jitter, random crop léger.

Temps de run estimé. Sur T4 (Colab Pro) : ~4–6 minutes par époque, soit ~2–3 heures pour un entraînement complet de 25 époques. Validation + métriques : ~5 minutes supplémentaires.

Métriques.

- Classification multi-label : Average Precision (AP) par catégorie, mean AP (mAP) sur les 26 catégories — métrique standard d'EMOTIC
- Régression VAD : Mean Absolute Error (MAE) et corrélation de Pearson par dimension (Valence, Arousal, Dominance), conformément au protocole de Kosti et al.
- Toutes les métriques sont calculées via `torchmetrics` (MultilabelAveragePrecision, MeanAbsoluteError, PearsonCorrCoef)

Composant 3 — Génération conditionnelle d'émotions

Approche retenue. Nous optons pour un fine-tuning LoRA de Stable Diffusion v1.5 via la librairie HuggingFace `diffusers`, conditionné sur le vecteur VAD cible. Ce choix repose sur un raisonnement d'élimination :

- Un cGAN entraîné from scratch nécessiterait un dataset bien plus large et un budget GPU incompatible avec Colab (les GANs sur images 256+ sont notoirement instables à entraîner).
- Un VAE conditionnel produirait des images trop floues en 3–4 semaines de développement.
- ControlNet est une option solide mais son fine-tuning complet dépasse nos ressources. LoRA, en revanche, ne modifie qu'un sous-ensemble de poids (rang 4 ou 8, soit ~1–4M de paramètres additionnels) et peut être entraîné en quelques heures sur T4.

Conditioning. Le vecteur VAD (3 dimensions continues, normalisées dans $[0, 1]$) est injecté dans le prompt textuel de Stable Diffusion via un template structuré : nous construisons un prompt du type `"a scene with [valence_descriptor], [arousal_descriptor] atmosphere"` où les descripteurs sont mappés à partir des valeurs VAD (ex : valence haute → "joyful", arousal bas → "calm"). Alternativement, nous expérimentons l'injection directe du vecteur VAD dans le cross-attention du UNet, en suivant l'approche de IP-Adapter, mais ceci est considéré comme une extension si le temps le permet.

Données d'entraînement. Un sous-ensemble de ~5 000 images EMOTIC avec leurs annotations VAD sert de paires (image, condition). Les images sont redimensionnées à 512×512.

Évaluation.

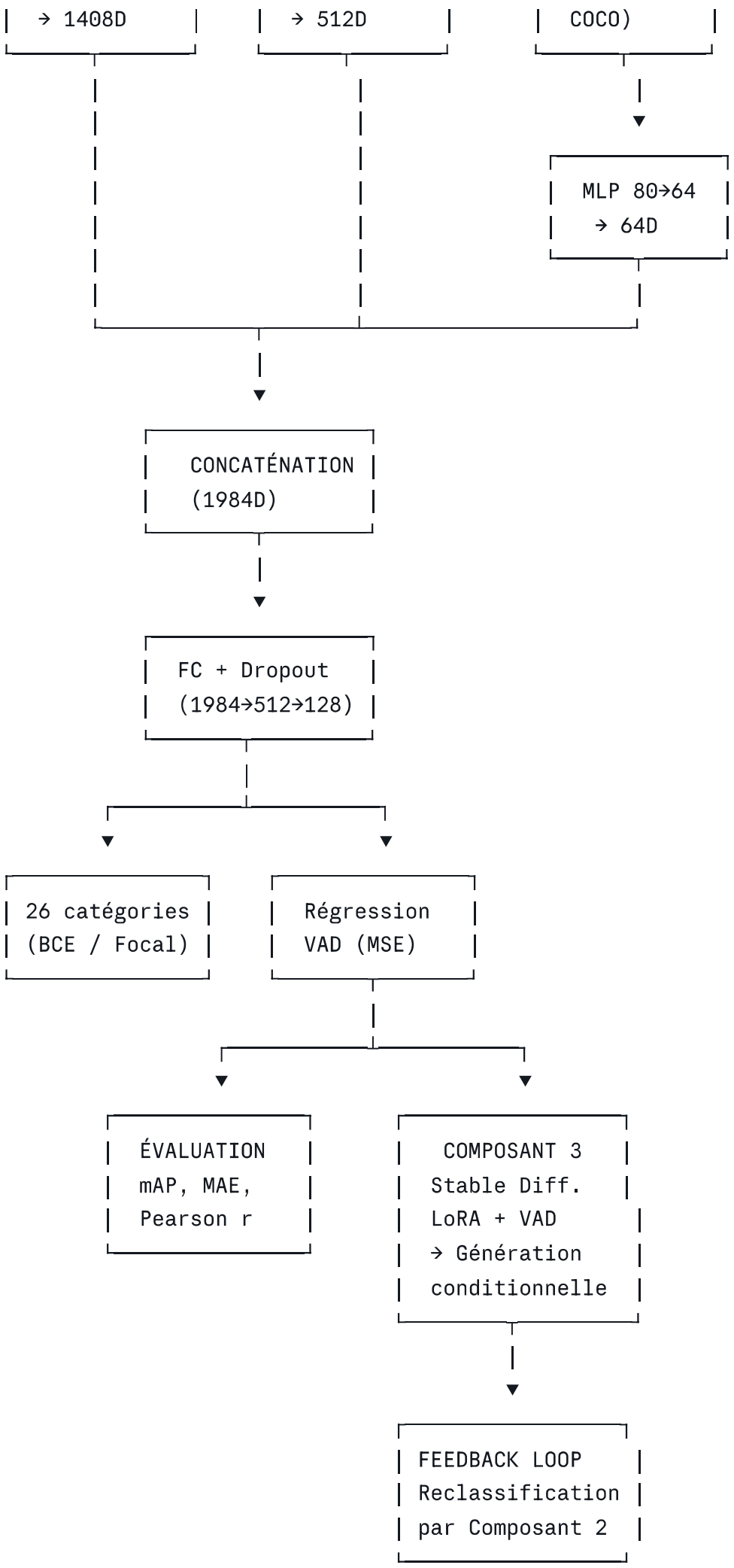
- Fréchet Inception Distance (FID) entre images générées et images réelles d'EMOTIC de même catégorie VAD (évaluation de la qualité visuelle)
- Score de reclassification : les images générées sont passées dans le Composant 2 entraîné, et nous mesurons la cohérence entre l'émotion cible (input de génération) et l'émotion prédite (output du classificateur). Ce feedback loop inter-tâches est au cœur de notre projet.
- Évaluation qualitative sur un panel réduit d'exemples visuels présentés sur le poster.

Temps de run estimé. Fine-tuning LoRA de SD 1.5 sur 5k images, 10 époques, batch size 1 avec gradient accumulation (4 steps) : ~4–6 heures sur T4. La génération d'images pour l'évaluation (~500 images) prend environ 30 minutes.

Librairies. `diffusers`, `transformers`, `peft` (pour LoRA), `torch-fidelity` ou `clean-fid` pour le calcul du FID.

Pipeline globale bout-en-bout

```
┌──────────────────────────────────────────────────────────────────────┐
│                          IMAGE D'ENTRÉE                              │
│                       (scène complète EMOTIC)                        │
└──────────┬──────────────────────┬──────────────────────┬─────────────┘
           │                      │                      │
           ▼                      ▼                      ▼
   ┌──────────────┐       ┌──────────────┐       ┌──────────────┐
   │  CROP PERSONNE│       │  IMAGE COMPLÈTE │    │   YOLOv8m    │
   │  (bbox EMOTIC)│       │   (224×224)     │    │  (inférence) │
   └──────────┬──────────┘  └──────────┬──────────┘  └──────────┬──────────┘
           │                      │                      │
           ▼                      ▼                      ▼
   ┌──────────────┐       ┌──────────────┐       ┌──────────────┐
   │ EfficientNet │       │  ResNet-18   │       │ Vecteur 80D │
   │     B2       │       │  (ImageNet)  │       │ (classes     │
```


→ 1408D
→ 512D
COCO)
MLP 80→64
→ 64D
CONCATÉNATION
(1984D)
FC + Dropout
(1984→512→128)
26 catégories
(BCE / Focal)
Régression
VAD (MSE)
ÉVALUATION
mAP, MAE,
Pearson r
COMPOSANT 3
Stable Diff.
LoRA + VAD
→ Génération
conditionnelle
FEEDBACK LOOP
Reclassification
par Composant 2

Pipeline développée sous PyTorch, entraînable intégralement sur Google Colab Pro (GPU T4, 16 Go VRAM). Temps total estimé : ~8–12 heures de GPU pour les Composants 2 et 3 combinés.